# Python: Standard Library Cheat Sheet

Fundamentals of the standard library — the "batteries included" modules that ship with Python itself, no installs needed. Each section shows the everyday core of one module or task; the full APIs live at docs.python.org. Language topics stay on the basics and advanced sheets.

## File & Console I/O

The basics sheet introduces `open()`; this is the fuller picture — appending, binary mode, and the standard streams.

```
import sys

# Console: print() writes to STDOUT; send errors to STDERR
print("normal output")                          # -> stdout
print("something went wrong", file=sys.stderr)    # -> stderr
sys.stdout.write("raw write\n")   # print() without separators/newline logic
# input() reads a line from stdin (see the getting-started sheet)

# The standard streams ARE ordinary file objects — the same type open()
# returns, pre-opened by the interpreter. The whole file interface works,
# and they fit anywhere a file object is expected (e.g. print(file=...)).
type(sys.stdout)             # <class '_io.TextIOWrapper'> (when a terminal)
sys.stderr.write("oops\n")   # 5 -> write() returns the chars written
sys.stdout.writable()        # True   (stdin: read-only; stdout/err: write-only)
sys.stdin.readable()         # True
# text = sys.stdin.read()    # read stdin until EOF — would sit waiting here
# for line in sys.stdin:     # ...or iterate it line by line, like any file
# Never close() them — the interpreter owns their lifetime.

# Text files: pass an explicit encoding, and let `with` close the file
with open("notes.txt", "w", encoding="utf-8") as f:   # w = write (TRUNCATES)
    f.write("first line\n")

with open("notes.txt", "a", encoding="utf-8") as f:   # a = append to the end
    f.write("second line\n")

with open("notes.txt", encoding="utf-8") as f:        # mode "r" is the default
    content = f.read()
content              # 'first line\nsecond line\n'

# Read line by line (memory-friendly: never loads the whole file)
with open("notes.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip())     # first line / second line

# Binary files: add "b" to the mode -> bytes in, bytes out, no encoding
# (the bytes type itself is covered on the basics sheet)
with open("blob.bin", "wb") as f:
    f.write(b"\x00\x01\xff")

with open("blob.bin", "rb") as f:
    data = f.read()
data                 # b'\x00\x01\xff'
len(data)            # 3

# Modes cheat: r read | w write (truncate!) | a append | x create, fail
```

```
# if it exists | +b modifiers combine: "rb", "a+", "xb"...

# Newlines: TEXT mode translates them, BINARY mode never touches them.
# Reading text converts \r\n (Windows) and \r to \n on ANY OS; writing
# text converts \n to the platform's ending (\r\n on Windows only).
with open("crlf.txt", "wb") as f:
    f.write(b"one\r\ntwo\n")          # raw bytes with a Windows line

with open("crlf.txt", encoding="utf-8") as f:
    f.read()          # 'one\ntwo\n'      -> \r\n silently became \n

with open("crlf.txt", "rb") as f:
    f.read()          # b'one\r\ntwo\n'   -> binary: untouched

# newline="" switches the translation OFF in text mode (csv needs this)
with open("crlf.txt", encoding="utf-8", newline="") as f:
    f.read()          # 'one\r\ntwo\n'

# The platform's own ending lives in os.linesep: '\n' here, '\r\n' on
# Windows — but code rarely needs it: just write '\n' and let text
# mode translate.
```

## Files, Directories & Paths

Everyday filesystem operations with `pathlib.Path`, `shutil`, and `tempfile`. (The `Path` object itself — construction, parts, joining with `/`, glob patterns — is covered on the advanced sheet.)

```
from pathlib import Path
import shutil

# Create directories
d = Path("data")
d.mkdir(exist_ok=True)             # no error if it already exists
(d / "sub").mkdir(parents=True, exist_ok=True)   # create nested dirs

# Quick one-shot file read/write (opens and closes for you).
# `/` on a Path JOINS segments: Path("data") / "notes.txt" is the path
# data/notes.txt — pure construction, nothing touches the disk yet.
# It's platform-independent: always WRITE /, and Path produces the
# OS's real separator underneath (backslash on Windows).
f = d / "notes.txt"
f.write_text("hello", encoding="utf-8")
f.read_text(encoding="utf-8")      # 'hello'

# Inspect
f.exists()                         # True
f.is_file(), d.is_dir()            # (True, True)
f.stat().st_size                   # 5 -> size in bytes

# List and search
sorted(x.name for x in d.iterdir())    # ['notes.txt', 'sub']
[x.name for x in d.glob("*.txt")]      # ['notes.txt']

# Copy, move/rename, delete
shutil.copy(f, d / "copy.txt")               # copy a file
(d / "copy.txt").rename(d / "renamed.txt")   # move / rename
(d / "renamed.txt").unlink()                 # delete a file
(d / "sub").rmdir()                # delete an EMPTY directory only
shutil.rmtree(d)                   # delete a whole tree — no recycle bin!
d.exists()                         # False
```

```
# Temp files/dirs that clean up after themselves
import tempfile
with tempfile.TemporaryDirectory() as tmp:
    Path(tmp, "scratch.txt").write_text("temp")
# leaving the with-block deleted tmp and everything in it

# Landmarks
cwd = Path.cwd()     # the process's current working directory
home = Path.home()   # the user's home directory
```

## Command-Line Arguments

```
import sys

# sys.argv holds the arguments as a list of STRINGS; [0] is the script.
# Running:  python myscript.py input.txt --fast
# gives:    sys.argv == ['myscript.py', 'input.txt', '--fast']

# argparse: declare the arguments, get parsing, validation, and --help
import argparse

parser = argparse.ArgumentParser(description="Process a file")
parser.add_argument("path")                          # required positional
parser.add_argument("--fast", action="store_true")   # boolean flag
parser.add_argument("--level", type=int, default=1)  # typed, with default

# In a real script:  args = parser.parse_args()   (reads sys.argv)
# An explicit list also works — used here to keep the example runnable:
args = parser.parse_args(["input.txt", "--fast", "--level", "3"])
args.path            # 'input.txt'
args.fast            # True
args.level           # 3  -> already an int, thanks to type=int

# Bad input never reaches your code: argparse prints usage and exits
# parser.parse_args([])       # error: the following arguments are
#                             # required: path  -> SystemExit(2)
# parser.parse_args(["-h"])   # prints full help and exits
```

## Environment Variables

```
import os

# os.environ is a dict-like view of the process environment
# os.environ["HOME"]            # e.g. '/Users/alice' — varies per machine
os.environ.get("API_KEY")       # None -> missing key, no error (like dict.get)
os.environ.get("API_KEY", "")   # ""   -> or supply your own default
# os.environ["API_KEY"]         # KeyError if not set ([] access raises)

# Setting: values must be STRINGS; child processes inherit them
os.environ["APP_MODE"] = "debug"
os.environ["APP_MODE"]          # 'debug'
# os.environ["PORT"] = 8000     # TypeError: str expected, not int
os.environ["PORT"] = "8000"     # numbers go in (and come out) as strings
int(os.environ["PORT"])         # 8000 -> convert on the way out

del os.environ["APP_MODE"]      # unset
"APP_MODE" in os.environ        # False
```

```
# Changes affect THIS process and its children only — never the shell
# that launched it; everything reverts when the process exits.
```

## JSON

```
import json

# Python -> JSON string: dumps ("dump to string"). The typical input is
# a DICT (any mix of the mapped types below works):
data = {"name": "Alice", "age": 30, "tags": ["admin"], "active": True}
json.dumps(data)   # '{"name": "Alice", "age": 30, "tags": ["admin"], "active": true}'

# JSON string -> Python: loads. A JSON object comes back as a real DICT
# (a top-level JSON array would come back as a list):
json.loads('{"x": 1, "ok": true, "note": null}')
# {'x': 1, 'ok': True, 'note': None}   -> a dict; true/null became True/None

# Any mapped type works at the TOP level too — e.g. a list
json.dumps([1, "two", True])   # '[1, "two", true]'
json.loads("[1, 2, 3]")        # [1, 2, 3]

# Mapping: dict<->object, list/tuple<->array, str<->string, int/float
# <->number, True/False<->true/false, None<->null. Sets don't fit:
# json.dumps({1, 2})   # TypeError: Object of type set is not JSON serializable

# Class instances don't serialize by themselves either...
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

u = User("Alice", 30)
# json.dumps(u)              # TypeError: Object of type User is not
#                            #            JSON serializable
# ...but their attribute dict does — vars(u) is u.__dict__ (see the
# advanced sheet's Introspection section):
json.dumps(vars(u))          # '{"name": "Alice", "age": 30}'
json.dumps(u, default=vars)  # same result — and default= also covers
                             # objects NESTED anywhere in the data

# Pretty-print with indent
print(json.dumps({"a": 1, "b": [2, 3]}, indent=2))
# {
#   "a": 1,
#   "b": [
#     2,
#     3
#   ]
# }

# Files: dump/load (no s) work on file objects directly
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)

with open("config.json", encoding="utf-8") as f:
    loaded = json.load(f)
loaded == data     # True -> clean round-trip

# Non-ASCII is escaped by default; ensure_ascii=False keeps it readable
```

```
json.dumps({"city": "città"})                      # '{"city": "citt\\u00e0"}'
json.dumps({"city": "città"}, ensure_ascii=False)  # '{"city": "città"}'

# GOTCHA: JSON object keys are ALWAYS strings — non-string keys convert
json.loads(json.dumps({1: "one"}))   # {'1': 'one'} -> the int key became '1'

# Other config formats: YAML is NOT in the stdlib (needs the
# third-party PyYAML); TOML is — tomllib (3.11+, parse-only), the
# format of pyproject.toml. INI files: configparser.
```

## Base64

Base64 represents arbitrary **bytes** as plain ASCII text — the standard way to move binary data through text-only channels (JSON, URLs, email). It is an encoding, **not encryption**: anyone can decode it, so it protects nothing.

```
import base64

# bytes in -> bytes out (the bytes type is on the basics sheet)
raw = b"hi there"
encoded = base64.b64encode(raw)   # b'aGkgdGhlcmU='
base64.b64decode(encoded)         # b'hi there'  -> clean round-trip

# Strings need the str<->bytes hop on BOTH ends
b64 = base64.b64encode("città".encode("utf-8")).decode("ascii")
b64                                    # 'Y2l0dMOg' -> a plain str now
base64.b64decode(b64).decode("utf-8")     # 'città'

# URL-safe variant: substitutes -_ for +/ (safe in URLs and filenames)
base64.b64encode(bytes([251, 255]))          # b'+/8='
base64.urlsafe_b64encode(bytes([251, 255]))  # b'-_8='
```

## Hashing

A hash is a fixed-length fingerprint of some bytes: one-way (no decoding back), deterministic, and any tiny change produces a completely different digest. Uses: integrity checks, deduplication, cache keys.

```
import hashlib

# The raw result is BYTES (32 for sha256): digest(). hexdigest() is the
# SAME value as hex text — 2 chars per byte, 64 total — which is what
# you print, store, and compare. hexdigest() == digest().hex()
hashlib.sha256(b"hi there").digest()[:4]   # b'\x9b\x96\xa1\xfe' (raw)
h = hashlib.sha256(b"hi there").hexdigest()
h            # '9b96a1fe1d548cbbc960cc6a0286668fd74a763667b06366fb2324269fcabaa4'
len(h)       # 64 -> always, regardless of the input's size
# (Below, [:16] only shortens output for THIS sheet — no other meaning.)

# The avalanche effect: one changed byte, entirely different digest
hashlib.sha256(b"hi therE").hexdigest()[:16]   # '6b2bf6242cbce8a0'

# Other algorithms, same interface. md5/sha1 are BROKEN for security —
# still fine for checksums and dedup, never for anything an attacker
# could exploit. Prefer sha256.
hashlib.md5(b"hi there").hexdigest()    # 'fd33e2e8ad3cb1bdd3ea8f5633fcf5c7'

# Feed data in chunks with update() — e.g. hashing a big file
inc = hashlib.sha256()
```

```
inc.update(b"hi ")
inc.update(b"there")
inc.hexdigest() == h    # True -> same as hashing it in one go

# PASSWORDS are a special case: plain sha256 is too fast to be safe.
# Use a slow, salted derivation like pbkdf2 (or scrypt):
dk = hashlib.pbkdf2_hmac("sha256", b"password", b"salt", 100_000)
dk.hex()[:16]           # '0394a2ede332c9a1'

# secrets: cryptographically strong random tokens (never use random!)
import secrets
secrets.token_hex(8)        # e.g. '162bae35215e0d6e' — new every call
secrets.token_urlsafe(8)    # e.g. 'O2k0llgqkIs'
```

## Dates & Times (`datetime`)

```
from datetime import date, datetime, timedelta

# The current moment — values obviously change every run:
today = date.today()        # a date:      e.g. date(2026, 7, 12)
now = datetime.now()        # a datetime:  date + time of day

# Fixed values are built by component
d = date(2026, 7, 12)
dt = datetime(2026, 7, 12, 9, 30)      # year, month, day, hour, minute
d.year, d.month, d.day      # (2026, 7, 12)
d.weekday()                 # 6 -> Monday is 0 ... Sunday is 6

# Arithmetic: timedelta is a DURATION; +/- shift dates, - gives gaps
d + timedelta(days=1)       # date(2026, 7, 13)
d + timedelta(weeks=2)      # date(2026, 7, 26)
deadline = datetime(2026, 12, 31, 23, 59)
(deadline - dt).days        # 172 -> subtraction yields a timedelta
d < date(2026, 12, 25)      # True -> comparisons just work

# Object -> string: strftime ("string format time")
dt.strftime("%Y-%m-%d")          # '2026-07-12'
dt.strftime("%d/%m/%Y %H:%M")    # '12/07/2026 09:30'
dt.isoformat()                   # '2026-07-12T09:30:00'

# String -> object: strptime ("string parse time") — the reverse
datetime.strptime("12/07/2026 09:30", "%d/%m/%Y %H:%M")
# datetime(2026, 7, 12, 9, 30)
datetime.fromisoformat("2026-07-12T09:30:00")   # shortcut for ISO strings

# All of the above are NAIVE (no timezone attached). For aware objects:
from datetime import timezone
utc_now = datetime.now(timezone.utc)    # tzinfo=UTC -> comparable across
                                        # zones; prefer aware in real apps
```

## Regular Expressions (`re`)

```
import re

text = "Order 66 shipped on 2026-07-12 to Alice"

# Write patterns as RAW strings (r"...") so \d survives untouched
re.search(r"\d+", text)     # <re.Match object; span=(6, 8), match='66'>
```

```
re.search(r"\d+", text).group()   # '66' -> first match, anywhere
re.match(r"\d+", text)      # None -> match() anchors at the START only
re.search(r"xyz", text)     # None -> no match is None: always check!

re.findall(r"\d+", text)    # ['66', '2026', '07', '12']  -> all matches
re.sub(r"\d", "#", text)    # 'Order ## shipped on ####-##-## to Alice'

# Groups: parentheses capture parts of the match
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", text)
m.group(0)           # '2026-07-12' -> the whole match
m.group(1)           # '2026'      -> first captured group
m.groups()           # ('2026', '07', '12')

# Named groups read better
m = re.search(r"(?P<year>\d{4})-(?P<month>\d{2})", text)
m.group("year")      # '2026'

# compile() the pattern once when reusing it; flags tweak the rules
word = re.compile(r"alice", re.IGNORECASE)
word.search(text).group()    # 'Alice'

# Mini pattern reference:
#   \d digit   \w word char   \s whitespace   .  any char
#   +  one or more   * zero or more   ? optional   {n} exactly n
#   ^  start   $ end   [abc] char set   (a|b) alternation
```

## Random Numbers (`random`)

Pseudo-random numbers for simulations, games, sampling. **Not for security** — tokens and passwords need `secrets` (see Hashing above).

```
import random

# In normal use, do NOT call seed(): Python seeds itself from OS
# entropy on first use, so every run gets a fresh, unpredictable
# sequence. Fix the seed ONLY when you WANT repeatability — tests,
# experiments, demos: same seed -> same numbers, forever (it is how
# the outputs below can be exact). A fixed seed is predictable by
# design; seed() with no argument re-seeds unpredictably from the OS.
random.seed(42)

random.random()          # 0.6394267984578837 -> float in [0.0, 1.0)
random.randint(1, 6)     # 1                  -> int, BOTH ends included
random.choice(["rock", "paper", "scissors"])  # 'scissors' -> one element

# sample(population, k): k DISTINCT picks, no repeats — a lottery draw.
# The population can be any sequence; range(1, 50) is the ints 1..49
random.sample(range(1, 50), 3)   # [18, 16, 15]

# uniform(a, b): one float anywhere between a and b (ends included) —
# the pick-from-a-range version of random(), which is locked to [0, 1)
random.uniform(1, 10)    # 2.255841356726295

# shuffle() reorders a list IN PLACE (mutates it) and returns None,
# so never write cards = random.shuffle(cards) — cards would be None!
cards = [1, 2, 3, 4, 5]
random.shuffle(cards)
cards                    # [4, 2, 3, 5, 1] -> same list, new order

# Want a shuffled COPY instead? Sample the whole list:
```

```
random.sample(cards, len(cards))   # [4, 1, 5, 3, 2] -> new list
cards                              # [4, 2, 3, 5, 1] -> original untouched
```

## Specialized Containers (`collections`)

Four upgrades over the built-in collections, each solving one recurring chore.

```
from collections import Counter, defaultdict, namedtuple, deque

# Counter: tallies anything iterable — the "counting dict"
votes = Counter(["yes", "no", "yes", "yes", "abstain"])
votes                     # Counter({'yes': 3, 'no': 1, 'abstain': 1})
votes["yes"]              # 3
votes["missing"]          # 0 -> missing counts are 0, never a KeyError
votes.most_common(2)      # [('yes', 3), ('no', 1)] -> top-k, ready sorted
Counter("mississippi").most_common(2)   # [('i', 4), ('s', 4)]

# Counters are mutable: increment freely (missing keys start at 0),
# feed in more items with update(), even add two Counters together
votes["late"] += 1        # works although "late" wasn't there
votes.update(["no", "no"])
votes                     # Counter({'yes': 3, 'no': 3, 'abstain': 1, 'late': 1})
Counter("aab") + Counter("abb")   # Counter({'a': 3, 'b': 3})

# defaultdict: a dict that CREATES the value on first access, using the
# factory you give it — kills the "if key not in d: d[key] = []" dance
groups = defaultdict(list)          # factory list -> default is a new []
groups["fruit"].append("apple")     # no KeyError: [] appeared, then append
groups["fruit"].append("pear")
dict(groups)              # {'fruit': ['apple', 'pear']}

# namedtuple: a tuple with NAMED fields — a lightweight, immutable record
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
p                         # Point(x=3, y=4) -> readable repr for free
p.x                       # 3 -> access by name...
p[0]                      # 3 -> ...but it's still a tuple (unpacking works)
# (need mutability, defaults, methods? -> dataclasses, advanced sheet)

# deque ("deck"): double-ended queue — fast appends/pops at BOTH ends,
# where lists are slow on the left
dq = deque([2, 3])
dq.appendleft(1)          # deque([1, 2, 3])
dq.append(4)              # deque([1, 2, 3, 4])
dq.popleft()              # 1 -> O(1); list.pop(0) shifts everything
dq                        # deque([2, 3, 4])
deque([1, 2, 3, 4], maxlen=3)   # deque([2, 3, 4], maxlen=3)
#                         -> bounded: keeps only the LAST 3 (rolling window)

# Stack and queue? No dedicated container needed:
stack = [1, 2]            # a STACK (LIFO) is just a list...
stack.append(3)           # push
stack.pop()               # 3 -> pop; both O(1) at the right end
q = deque([1, 2])         # a QUEUE (FIFO) is a deque...
q.append(3)               # enqueue at the right
q.popleft()               # 1 -> dequeue at the left (never list.pop(0)!)
# queue.Queue/LifoQueue exist too, but they're THREAD-SAFE pipes for
# producer/consumer concurrency, not everyday containers. Priority
# queues: heapq.
```

## Running External Commands (`subprocess`)

```
import subprocess
import sys

# run() starts a program and waits for it. Pass the command as a LIST:
# no shell parsing, no quoting headaches, no injection risk.
# sys.executable is a string: the absolute path of the interpreter
# currently running (e.g. '/usr/local/bin/python3.14'). Launching it
# as the child means "this exact same Python" — version and venv
# included — so these examples run anywhere. The pattern is the same
# for any program: ["git", "status"], ["ls", "-la"], ...
result = subprocess.run(
    [sys.executable, "-c", "print('hi from a child process')"],
    capture_output=True,    # collect stdout/stderr instead of inheriting
    text=True,              # decode bytes -> str
)
result.returncode        # 0 -> the process's exit code; 0 means success
result.stdout            # 'hi from a child process\n'
result.stderr            # ''

# A failing command does NOT raise by default — inspect returncode...
bad = subprocess.run([sys.executable, "-c", "raise SystemExit(3)"])
bad.returncode           # 3
# ...or pass check=True to turn failures into exceptions:
# subprocess.run([...], check=True)
#   -> CalledProcessError: Command '[...]' returned non-zero exit status 3.

# The command must be a REAL executable on PATH, and availability
# varies by OS: dir is a binary on Linux, missing on macOS (use ls),
# and a cmd.exe BUILT-IN on Windows — built-ins need their shell:
# subprocess.run(["dir"])             # macOS: FileNotFoundError
# subprocess.run(["cmd", "/c", "dir"])  # Windows: the way to a built-in

# Avoid shell=True (one big string parsed by the shell): it reintroduces
# exactly the quoting and injection problems the list form avoids.
```

## Logging

`print` is for program *output*; `logging` is for *diagnostics* — messages with severity levels that can be filtered, timestamped, and redirected without touching the calling code.

```
import logging

# Five levels: DEBUG < INFO < WARNING < ERROR < CRITICAL.
# With NO configuration at all: only WARNING and above are shown (the
# classic surprise is logging.info() being silently dropped), in the
# default format "LEVEL:name:message" with name "root":
# logging.warning("disk almost full")   # WARNING:root:disk almost full

# Configure ONCE, at program start (only the first call takes effect;
# force=True replaces an existing configuration)
logging.basicConfig(level=logging.INFO,
                    format="%(levelname)s %(name)s: %(message)s",
                    force=True)

log = logging.getLogger(__name__)   # idiom: one logger per module

log.info("starting up")             # INFO __main__: starting up
```

```
log.warning("disk almost full")     # WARNING __main__: disk almost full
log.debug("not shown")              # DEBUG < INFO -> dropped
log.error("write failed")           # ERROR __main__: write failed

# Messages go to STDERR by default (stdout stays clean for output);
# add filename="app.log" in basicConfig to log to a file instead.

# Pass values as ARGUMENTS, not in an f-string: the message is only
# formatted if it will actually be emitted (cheap when filtered out)
user = "alice"
log.info("user %s logged in", user)   # INFO __main__: user alice logged in

# Inside an except block, exception() = error() + the full traceback
try:
    1 / 0
except ZeroDivisionError:
    log.exception("division blew up")
# ERROR __main__: division blew up
# Traceback (most recent call last): ... ZeroDivisionError: division by zero

# The format string is built from %(...)s placeholders — common ones:
#   %(asctime)s    timestamp          %(levelname)s   level
#   %(name)s       logger name        %(message)s     the message
#   %(filename)s   source file        %(lineno)d      line number
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",   # strftime codes (see datetime above)
    force=True,
)
log.info("timestamped")   # e.g. 2026-07-12 11:19:09 INFO __main__: timestamped
```

## Iterator Tools (`itertools`)

Lazy building blocks for loops — like generators (basics sheet), they yield one item at a time, so they handle huge or even infinite streams. Everything below returns an iterator; wrap in `list()` to see the items.

```
from itertools import (chain, count, islice, product,
                       permutations, combinations, zip_longest, batched)

# chain: several iterables, one seamless loop (nothing is concatenated)
list(chain([1, 2], (3, 4), "ab"))    # [1, 2, 3, 4, 'a', 'b']

# count: 10, 11, 12, ... forever — islice takes a slice of ANY iterator
# (generators have no [:5]; islice is how you "slice" them)
list(islice(count(10), 5))           # [10, 11, 12, 13, 14]

# product: the cartesian product — nested loops in one call
list(product("AB", "12"))    # [('A','1'), ('A','2'), ('B','1'), ('B','2')]

# permutations: ordered arrangements | combinations: unordered picks
list(permutations([1, 2, 3], 2))
# [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
list(combinations([1, 2, 3], 2))
# [(1, 2), (1, 3), (2, 3)]    -> no (2, 1): order doesn't matter here

# zip_longest: like zip, but pads to the LONGEST input
# (plain zip stops at the shortest — see the basics Loops section)
list(zip_longest([1, 2, 3], "ab", fillvalue="-"))
# [(1, 'a'), (2, 'b'), (3, '-')]
```

```
# batched (3.12+): fixed-size chunks — the last may be shorter
list(batched([1, 2, 3, 4, 5], 2))    # [(1, 2), (3, 4), (5,)]
```

## CSV Files (`csv`)

```
import csv

# Always open csv files with newline="". The csv module writes its own
# row endings: \r\n by default (the RFC 4180 / Excel convention — most
# parsers happily accept plain \n too, see below). Text mode would
# translate AGAIN (see File I/O): on Windows every row would become
# \r\r\n — the classic "blank line after each row in Excel" bug — and
# reading can corrupt newlines inside quoted fields. newline="" means
# only the csv module decides line endings, never the file mode.
rows = [["name", "age"], ["Alice", 30], ["Bob", 25]]
with open("people.csv", "w", encoding="utf-8", newline="") as f:
    csv.writer(f).writerows(rows)        # writerow() does a single one
# people.csv now contains:
#   name,age
#   Alice,30
#   Bob,25

# Reading: EVERY field comes back as a string — convert numbers yourself
with open("people.csv", encoding="utf-8", newline="") as f:
    data = list(csv.reader(f))
data     # [['name', 'age'], ['Alice', '30'], ['Bob', '25']]
#                                    ^^^^ '30', not 30

# DictReader/DictWriter: rows as dicts, keyed by the header row
with open("people.csv", encoding="utf-8", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["age"])   # Alice 30 / Bob 25

with open("people.csv", "w", encoding="utf-8", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "age"])
    w.writeheader()                      # writes: name,age
    w.writerow({"name": "Carol", "age": 35})   # writes: Carol,35

# Quoting is automatic when a value contains the delimiter or quotes
import io
buf = io.StringIO()                      # an in-memory text file
csv.writer(buf).writerow(["hi, there", 'say "hi"'])
buf.getvalue()       # '"hi, there","say ""hi"""\r\n'

# Prefer plain \n row endings (e.g. cleaner git diffs)? Override them —
# the newline="" advice still applies, so nothing re-translates behind
# your back:
buf = io.StringIO()
csv.writer(buf, lineterminator="\n").writerow(["a", "b"])
buf.getvalue()       # 'a,b\n'

# Different separator: pass delimiter= to writer AND reader alike
with open("semi.csv", "w", encoding="utf-8", newline="") as f:
    csv.writer(f, delimiter=";").writerows([["name", "age"], ["Alice", 30]])
# semi.csv now contains:
#   name;age
#   Alice;30
with open("semi.csv", encoding="utf-8", newline="") as f:
    list(csv.reader(f, delimiter=";"))   # [['name', 'age'], ['Alice', '30']]
```